Model Provenance Management in MLOps Pipeline

Songzhu Mei*
Science and Technology on Parallel and Distributed Laboratory, National University of Defense Technology

Cong Liu
Information Center, Logistic Support Department of CMC

Qinglin Wang*
Science and Technology on Parallel and Distributed Laboratory, National University of Defense Technology

Huayou Su
Science and Technology on Parallel and Distributed Laboratory, National University of Defense Technology

ABSTRACT

*Machine learning engineering is an important technology that has attracted the attention of academia and industry in the past two years. For AI to become a productivity of enterprises, it must be engineered to solve the problems of model development, deployment, management, prediction, and other lifecycle management, and MLOps system is the most concerned technology in the current machine learning engineering process. This paper starts from the training evolution process of intelligent computing models in machine learning engineering, focuses on the influence of data, training algorithms, and requirement indicators on the model evolution process, and gives the definition of Model Provenance and its evolutionary correlation elements. Based on the concept of model lineage, this paper designs a method to manage the model evolution process in the MLOps system and implements the related system.

CCS CONCEPTS

• **Artificial intelligence**; • **Machine learning**; • **Software creation and management**;

KEYWORDS

Model provenance, MLOps, Machine learning engineering, Artificial Intelligence

ACM Reference Format:
Songzhu Mei*, Cong Liu, Qinglin Wang, and Huayou Su. 2022. Model Provenance Management in MLOps Pipeline. In *2022 The 8th International Conference on Computing and Data Engineering (ICCDE 2022), January 11–13, 2022, Bangkok, Thailand.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3512850.3512861

1 INTRODUCTION

In recent years, machine learning technology represented by deep learning has achieved great success in many fields, including computer vision, natural language processing and big data analysis and mining. For example, deep learning methods have been outperforming traditional object recognition and detection methods in the ISLVRC computer vision competition since 2012, and are gradually approaching and surpassing human levels. For example, the large scale pre-training model represented by GPT-3 shows a similar level of human ability in computational tasks related to natural language processing. However, in the actual production environment, it is often difficult to achieve the expected performance of intelligent computing applications based on machine learning model. On the one hand, machine learning models are usually very complex and not explicable, which makes the behavior of models difficult to predict and test. On the other hand, the development, deployment and continuous improvement of intelligent computing applications do not rely solely on machine learning models. According to the research of Google [1], in the actual intelligent computing application, machine learning model usually only accounts for a small proportion in the system, and there are a lot of infrastructure and processes around the model to support the application evolution. As applications evolve, multiple sources of technical debt accumulate, leading to the behavior of applications deviating from users' expectations of the machine learning model itself. Some of the sources of technical debt are closely related to data dependence, model complexity, reproducibility, testing, monitoring and external response changes. It is difficult to adapt to the changes faced by the application system simply relying on model training process or model algorithm improvement. Gartner's report estimates that through 2020, 80% of AI projects will remain alchemy, run by wizards whose talents will not scale in the organization.

For a long time, because of machine learning models are considered as core functions of intelligent computing application systems, researchers and developers focus on the programming, training and optimization of a model. Our impression of machine learning is often get a bunch of excel/CSV data, do some model experiments with notebook, and evaluate final prediction results. But we probably don't have an insight of how this prediction will be used or what the business impact will be. It's easy to end up with machine learning projects stuck in the lab, keep doing proof-concept tests without successfully translating into a real-world production. In recent years, enterprises have paid more and more attention to the implementation of machine learning projects. They have better understanding of business and application processes, and more and more models have been deployed in real business scenarios. However, when the business actually started running, there would be a variety of requirements feedback on the model, and engineers would begin to do continuing iterative development and frequent deployment online. With the development of business, more and

*Both Songzhu Mei and Qinglin Wang are corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCDE 2022, January 11–13, 2022, Bangkok, Thailand
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9571-7/22/01…$15.00
https://doi.org/10.1145/3512850.3512861

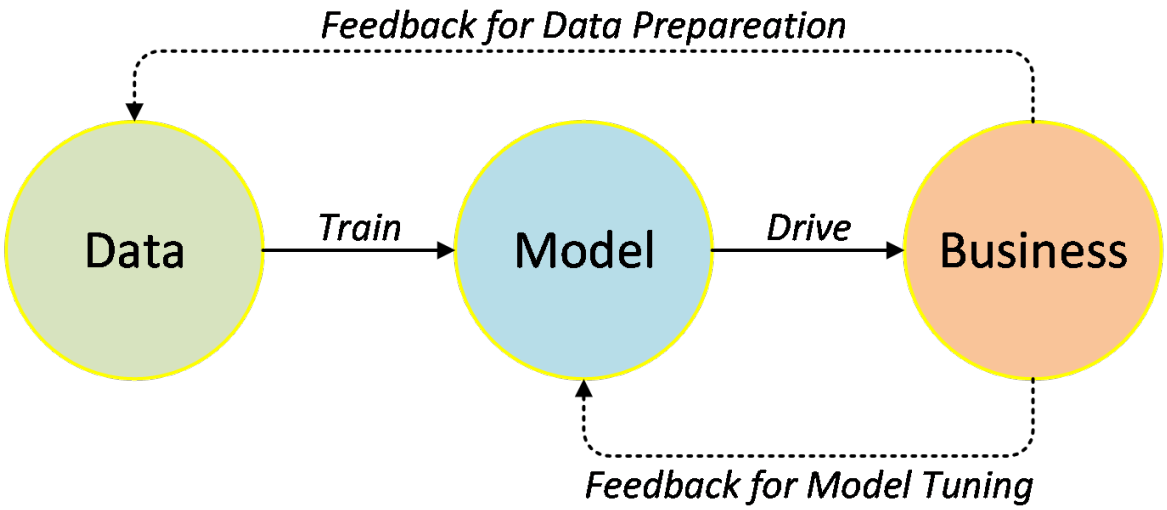

Figure 1: The Interactive relationship between data, model, business in intelligent computing applications

more model application scenarios, managing and maintaining a large number of models and intelligent application systems has become a real challenge. In order to eliminate the technical debt in the intelligent computing application system, improve the adaptability of machine learning model to the actual production environment, and accelerate the transformation efficiency of machine learning model to intelligent computing application system, MLOps came into being. MLOps is DevOps in the era of machine learning. It plays as a connector between the model building team and the operation team, and establishes a standard model development, deployment and operation and maintenance process, so that enterprises can better utilize the ability of machine learning to promote business growth.

This paper focuses on the integration and transformation process between machine learning model and intelligent computing application. We establishes a model provenance management mechanism, which can provide the ability of fast model branching and training iteration for intelligent computing application system development. Finally, a model lineage management system for MLOps system is designed and implemented, which obtains the code, dataset, model and training/deployment status information in MLOps system through the interface, and constructs model lineage database and model performance database, which provides a new technical support for the engineering application of machine learning.

2 BACKGROUND

2.1 MLOps and AI Engineering

Unlike traditional software systems that focus only on business logic code, a typical intelligent computing application contains three elements: data, model, and business code. The model, as the core of the intelligent computing application, needs to be influenced by both data and business changes. The relationship between the three is shown in Figure 1. Data is used for model training, and the model is used to drive business operations after training is completed. Each change in feedback during this process results in a change in the performance of the intelligent computing application.

With the development and improvement of machine learning and deep learning technologies, artificial intelligence has moved out of the laboratory and into the production environment. However, in the process of applying machine learning models to actual production, there is a model drift problem that cannot be ignored, which makes the performance of machine learning models deviate from people's expectations (i.e., business goals). Model drift usually refers to the deterioration of model predictions over time, and is divided into two categories: concept drift and data drift. The goal of MLOps is to introduce the traditional software engineering concept of DevOps into the field of machine learning, and to support the continuous integration and continuous release of machine learning data, models and business through an automated or semi-automated workflow, so as to provide a better understanding of the business, models and data. The goal is to build an effective linkage mechanism between changes in any one element and the other two elements, and to promote the continuous improvement of the performance and quality of intelligent computing applications.

2.2 Model Provenance

In the field of data science, data provenace has always been a research point that has received attention. The data is described throughout the process, and the process evolved over time, its application is wide, including data quality evaluation, data check, data recovery, and data reference. The data is generally divided into data evolution processes between different data sources and the data evolution process inside the same data source. The foregoing mentioned that the data played a basic role in the intelligent computing application development process, and the changes in data will result in changes in model performance to a certain extent. In order to meet the changes in business needs, the machine learning algorithm engineers will adjust the data set or optimization model

algorithm for the case of model drift. This process is similar to the data science in the field of traditional data science, so this article uses the model provenance to express the process of generating and evolving the application of intelligent computing applications. Model Billets can be used to evaluate model quality, find optimal model versions of model optimization for specific business demand changes.

3 DESIGN AND IMPLEMENTATION

3.1 Affecting Factors of Model Provenance

In machine learning, models are at the heart of the most attention. A machine learning model is often described as a file that has been trained to recognize certain types of patterns. From this definition, we can see several typical properties of machine learning models.

- Persistability. A model is stored as a file, i.e., it can exist for a long time and can be parsed according to certain rules for model reuse.
- Need to be trained. A model is a product of training, and the ability of a model to recognize certain types of patterns comes from the training process.
- Ability to be bounded. At least so far, models in machine learning can only be used to recognize specific patterns and do not yet have general intelligence.

The persistence of machine learning models is the basis of model provenance. By deserializing model files, a model can be used for inference tasks or for further training (e.g., model update, migration learning, etc.).

The process of model training is the driver of model provenance. The essence of training is the process of extracting sample data features and revising the parameters in the model so that the model can better recognize the patterns present in the data. The model training process usually introduces a series of optimization methods to accelerate the process of minimizing the loss function, and different training optimization methods may have different effects on the model performance and training efficiency.

The model capability boundary can be used as a marker to determine the different stages of model provenance. The capability boundary is usually specified by a specific business goal, and the change of model capability boundary (including functional boundary, performance boundary, robustness boundary, etc.) is the process of the model reaching a specific metric.

It can be considered that the evolution of a machine learning model is a process in which indicators, data, and training methods work together as shown in Figure 2. Indicators set the target boundaries for model training evolution, data are the basis for whether the model training process can achieve the indicators, and training methods are the determinants of whether the model training process is efficient.

The model provenance may continue to change with changes in indicators, data, and training methods, especially when changes in indicators may cause the model to branch. For example, when functional indicators change, which may result in existing data as well as training methods no longer being applicable to the new indicators, it may cause fundamental changes in the model, resulting in model branching. In contrast, simple changes in dataset expansion or labeling methods, as well as changes in training methods, may affect the achievement of performance indicators more.

3.2 Design and Implementation

3.2.1 Design Objectives. The core objective of model provenance management is to manage the evolution of a model developed and trained in a machine learning task. However, considering that model evolution is influenced by several relatively independent but closely related factors, managing and maintaining model versions alone does not reflect the process of model evolution, and model provenance should include all factors that influence the process of model evolution. Therefore, model provenance management should cover the following objectives:

- Training requirements and indicators management. The first goal of model family management is to manage the requirements and objectives of machine learning tasks. On the one hand, the requirements of machine learning tasks directly affect the acquisition of data sets and the selection of model structures (e.g., regression or classification, decision trees or neural networks, convolutional or recurrent neural networks, etc.), and on the other hand, the requirements of metrics determine the termination conditions of the model training process. Therefore, the management of model training requirements and indicators requirements is the starting point of model provenance management, and runs through the whole life cycle of model evolution.
- Dataset management. Datasets frame the upper bound of the final effect that can be achieved for model training, and in the existing machine learning theory, models cannot break through the feature constraints in datasets. In the last two years, more and more data scientists and data engineers have adjusted the process of engineering machine learning from model-centric to data-centric, and it can be said that dataset management is the most important factor of model provenance management.
- Training algorithm management. On the one hand, for the same dataset and the same model structure, selecting different training algorithms may lead to different training results, such as model convergence speed, precision, and recall rate may be different. On the other hand, due to the different requirements and indicators of the target platform for model application, a series of new optimization methods are also introduced, such as compression methods in the field of deep learning such as quantization methods, all of which can lead to changes in the model evolution path. Therefore, training algorithm management is also an important aspect of model provenance management to be considered.
- Model performance management. Model performance reflects the model training effect, especially in the actual production environment model performance changes often become an important reason for the adjustment of model training needs and indicators requirements, and is the main contributor to the formation of continuous model evolution, so it is necessary to incorporate model performance management into the scope of model provenance management.

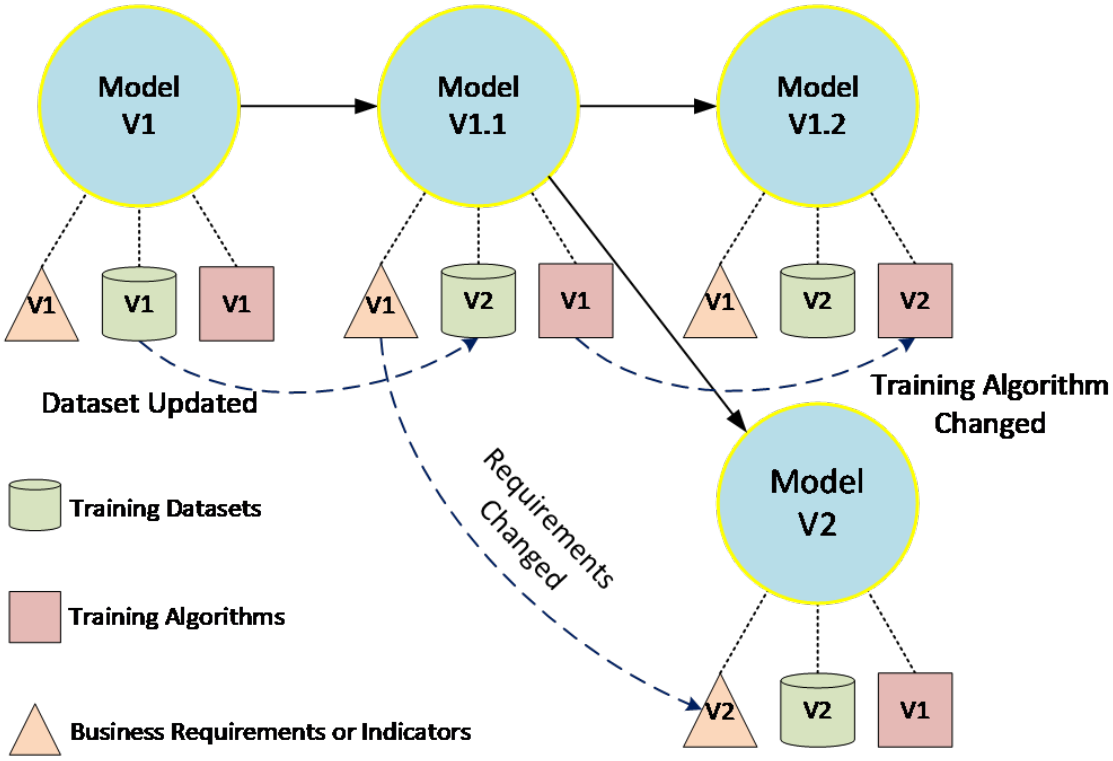

Figure 2: The affecting factors of the model provenance

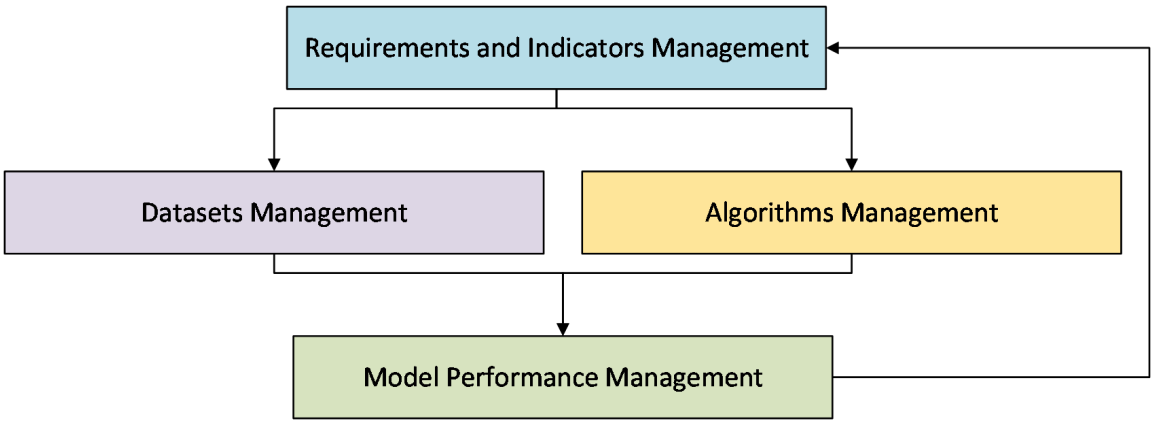

Figure 3: Objective system of model provenance management

Overall, the main objective system of model provenance management is shown in Figure 3, which is the need to build a process with feedback including requirements and indicators management, data set management, training algorithm management, and performance management to ensure the traceability of model evolution in machine learning tasks, and to support the migration of model evolution process from one class of machine learning tasks to another class of tasks to achieve evolutionary acceleration of model training.

3.2.2 Architecture of Model Provenance Management System. In order to achieve the above object, we designed and implemented a software system specifically designed to model the model of the model, which can be embedded in a typical MLOPS workflow, providing a comprehensive model of the MLOPS pipeline management, and the ability of the model dielectric analysis. The main technical architecture of the system is shown in Figure 4

The model provenance management system consists of two subsystems, the model provenance maintaining sub-system and the runtime maintaining sub-system. The former is responsible for

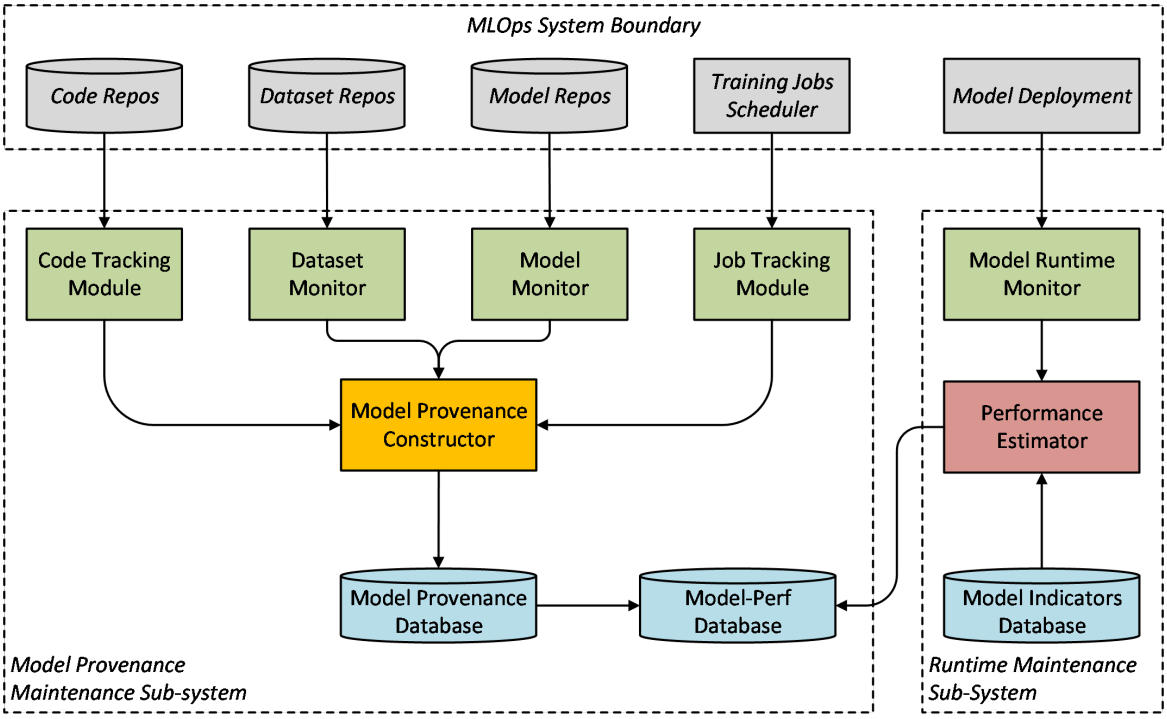

Figure 4: The architecture of model provenance management system

building and maintaining the model provenance information in the MLOps system, and keeping the model provenance information synchronized with the process of model development and evolution in MLOps. The latter is responsible for maintaining the model indicators requirements, obtaining the model runtime status, evaluating the model performance according to the indicators requirements, and maintaining the data of the model performance changes in that version. The model provenance management system is independent of the MLOps system and interacts with the MLOps system in a less invasive way through interfaces and callback functions. For the whole model provenance management system, the former is the core sub-system, and the following focuses on the design ideas of the model provenance maintenance sub-system.

The model provenance maintenance sub-system consists of the following main components:

- Code tracking module. It collects the model code repository in machine learning tasks through the interface in the MLOps system, tracks the version updates of model code and training algorithm code, and submits the code version information to the model provenance constructor for model provenance construction.
- Data set monitor. Through the interface in MLOps system, we monitor the changes of dataset, including dataset expansion, dataset widening, data annotation, etc., and record the dataset version and mark the changes between different dataset versions in conjunction with the dataset selection of training jobs. For MLOps systems with dataset version management capability, the dataset versions are obtained directly from the system.
- Job tracking module. It tracks the model training and validation jobs running in the MLOps system, focusing on obtaining the dataset used for training, code version, and training process status, and submits the relevant information to the model family constructor for model family construction.
- Model monitor. In conjunction with the job tracking module, the model monitor monitors the model release in the MLOps system. The model release is generally released after the model has been trained and validated, and the model monitor keeps an eye on the additions and deletions of the model and records this information for model family construction.
- Model provenance constructor. By integrating and analyzing the information collected from the above four modules, the model provenance is continuously constructed and persisted to the model family database.
- Model provenance database. A graph database for storing model provenance information. From the analysis in subsection 3.1, it is easy to find that the model provenance is a directed acyclic graph, so it is stored through the graph database.
- Model performance database. In conjunction with the performance estimator in the runtime maintenance sub-system, the performance of a particular version of the model is persisted over time. A temporal database is used to implement the storage of performance data and to associate it with a specific model version in the model provenance database.

4 RELATED WORK

Some work have been done in machine learning engineering. In [2], the authors define and identify the concept of "Data Cascades", and highlight an under-explored but significant new research path for the field in creating interfaces, processes, and policy for data excellence in AI. Liu et. al [3] demonstrate a working example of training and deploying a machine learning model for the application of detecting software repository code vulnerability. Goyal [4] delves into Explaining MLOps, Differentiating MLOps with DevOps, Working, and implementation of MLOps, Reason for MLOps Adoption, Its benefits, Challenges to MLOps, Standard practices, and tools used for operations. In [5], the survey shows that practitioners face challenges at each stage of the deployment. The goal of this paper is to layout a research agenda to explore approaches addressing these challenges. Renggli et. al. show that different components of a typical MLOps pipeline can be efficiently designed, providing both a technical and theoretical perspective by performing a joint analysis of the impact of well-known data quality dimensions and the downstream machine learning process [6]. In [7], a potential implementation of MLOps pipeline was presented using open source tools and platforms. The authors study integration between two organizations in detail and address scaling of AI/ML to multi-organization context in [8]. The paper [9] offers a concise definition of MLOps and AI Software Sustainability and outlines key challenges in its pursuit. Zhou et. al. built a functional ML platform with DevOps capability from existing continuous integration (CI) or continuous delivery (CD) tools and Kubeflow, constructed and ran ML pipelines to train models with different layers. The authors study the importance of MLOps in the context of data scientists' daily activities and set up an MLOps pipeline is a natural step to take, when an organization takes the step from ML as a proof-of-concept to ML as a part of nominal activities.

5 CONCLUSIONS

This paper first analyzes the design risks and challenges in the process of engineering machine learning, focusing on the many uncontrollable aspects of model-centric machine learning tasks from development to use in production environments. After that, the idea of implementing model lineage management in the process of machine learning application development is proposed, and the definition and influencing factors of model lineage are given. Finally, a model lineage management system for MLOps system is designed and implemented, which obtains the code, dataset, model and training/deployment status information in MLOps system through the interface, and constructs model lineage database and model performance database, which provides a new technical support for the engineering application of machine learning. We will present our detail research in future paper.

ACKNOWLEDGMENTS

This work is sponsored in part by the National Key Research & Development Program of China under Grant No. 2018YFB2101100, and Key Research Project of Logistic Support Department of CMC under grant BS220R007.

REFERENCES

[1] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. 2015. Hidden Technical Debt in Machine Learning Systems. NIPS.

[2] Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P.K., & Aroyo, L. 2021. "Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI. Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems.

[3] Liu, Y., Ling, Z., Huo, B., Wang, B., Chen, T., & Mouine, E. 2020. Building A Platform for Machine Learning Operations from Open Source Frameworks. IFAC-PapersOnLine, 53, 704-709.

[4] Akshita Goyal. 2020. Machine Learning Operations. International Journal of Information Technology Insights & Transformations Vol 4 No 2 (2020)

[5] Paleyes, A., Urma, R., & Lawrence, N.D. 2020. Challenges in Deploying Machine Learning: a Survey of Case Studies. ArXiv, abs/2011.09926.

[6] Renggli, C., Rimanic, L., Gurel, N.M., Karlavs, B., Wu, W., & Zhang, C. 2021. A Data Quality-Driven View of MLOps. ArXiv, abs/2102.07750.

[7] Ruf, P., Madan, M., Reich, C., & Ould-Abdeslam, D. 2021. Demystifying MLOps and Presenting a Recipe for the Selection of Open-Source Tools. Applied Sciences.

[8] Granlund, T., Kopponen, A., Stirbu, V., Myllyaho, L., & Mikkonen, T. 2021. MLOps Challenges in Multi-Organization Setup: Experiences from Two Real-World Cases. 2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI (WAIN), 82-88.

[9] Tamburri, D.A. 2020. Sustainable MLOps: Trends and Challenges. 2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), 17-23.

[10] Zhou, Y., Yu, Y., & Ding, B. 2020. Towards MLOps: A Case Study of ML Pipeline Platform. 2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE), 494-500.

[11] Mäkinen, S., Skogström, H., Laaksonen, E., & Mikkonen, T. (2021). Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help? 2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI (WAIN), 109-112.